

CS683 PA-1, Task 2: Matrix Multiplication

Tasks 2A, 2B, 2C — Software Prefetching, SIMD, and the Combined Kernel

Methodology

All measurements are single-precision SGEMM in the “NT” layout ($C[i][j] = \sum_p A[i][p] B[j][p]$), both operands row-major with the contraction dimension K contiguous), square matrices $M=N=K$.

Hardware. A Intel x86 architecture core CPU.

Timing. This machine shows substantial run-to-run variance (turbo/thermal state), up to 30–40% for an otherwise-identical configuration measured twice in a row. A single timed invocation is therefore not trustworthy for the fine-grained comparisons this report makes (prefetch distance, locality hint). Every timing number below is the **median of 5 independent process invocations**, each itself the median of several in-process repetitions (warmup discarded).

Hardware performance-counter values (L1D/L2/LLC misses, instruction counts, `sw_prefetch_access`) are far less sensitive to this noise — they count actual retired events rather than depending on clock speed — so those are taken from a single `perf stat`-wrapped run per configuration.

Speedup. $\text{Speedup} = \frac{\text{Execution time of the baseline}}{\text{Execution time of the optimized kernel}}$, equivalently the ratio of achieved GFLOP/s since FLOP count is fixed for a given size.

1 Task 2A — Software Prefetching

The kernel under test is `matmul_prefetch.cpp`: an AVX2 register-tiled (4×2) dot-product micro-kernel, M/N cache-tiled (256-row/column panels) so the tile’s A - and B -panels stay resident in L2 across the micro-kernel calls inside one tile. An `enable` flag (compiled in, default on) lets the prefetch calls be switched off without changing anything else about the kernel, which is what isolates the prefetch-specific contribution below from the tiling it sits on top of.

1.1 Speedup vs. Matrix Size

Trend. Speedup over naive grows with N up to ~ 1024 (tiling gives the kernel more reuse to exploit as the working set outgrows L1), dips at 1536 (the fixed 256-row tile panel, at $256 \times 1536 \times 4 \text{ B} = 1.5 \text{ MB}$, is large enough relative to this machine’s L2 that tile-panel reuse degrades before growing again at 2048 once the absolute FLOP count dominates fixed overheads). At *every* size, software prefetch is a net loss on top of tiling: 6–24% slower than the same tiled kernel with the prefetch calls simply removed.

N	naive	tilde+SIMD, no SW prefetch	tilde+SIMD+SW prefetch
128	1.000	3.409	2.606
256	1.000	4.783	4.339
512	1.000	5.288	4.526
1024	1.000	5.704	5.081
1536	1.000	3.658	3.429
2048	1.000	6.159	5.605

Table 1: Speedup over `matmul_naive`, `dist=512`, `hint=NTA`, `tile=256`.

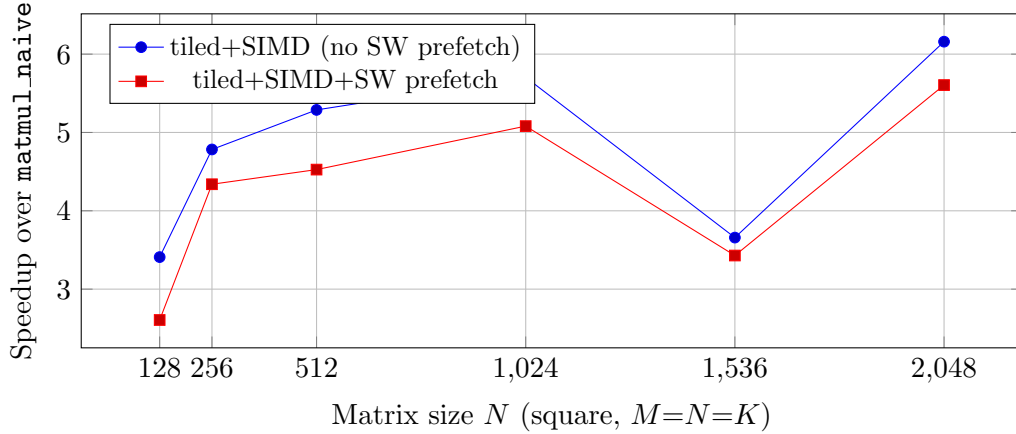


Figure 1: 2A: speedup vs. matrix size, with and without software prefetch.

1.2 Speedup vs. Prefetch Distance

Best distance: ~ 128 – 256 floats (2–4 cache lines ahead). Both very short distances (the line has too little lead time to arrive before it is consumed) and very long distances (the prefetched line risks being evicted again before the K-loop actually reaches it, or simply wastes issue slots earlier than needed) underperform this middle ground.

1.3 Speedup vs. Cache Fill Level (Locality Hint)

Best hint: `NTA`, with `T1` close behind; **`T0` is clearly worst** (26–32% Slower than `NTA`/`T1`). This is precisely what the L1D-miss counters only partly explain: `T0` and `NTA` both give the *real* loads very low L1 miss rates (the prefetch has already placed the line), while `T1`/`T2` (which target L2, not L1) leave the real load to We are still experiencing L1 misses as we move towards a fast L2 hit. Yet at `T0` a low number of L1 misses was also achieved. rate, it is still the slowest overall — this micro-kernel’s accumulators are held in an *array* (`_m256 acc[MR][NR]`) that, at these compiler flags, is not proven alias-free and round-trips through the stack/L1 every K-step.

1.4 Hardware Prefetcher On/Off

Using `wrmsr -p 0 0x1a4`, bits 0–3, pinned to the same P-core for both states.

Distance (floats)	Speedup vs. naive
8	5.065
16	5.339
32	5.197
64	5.303
128	5.593
256	5.612
512	5.568
1024	5.231

Table 2: $N=2048$, hint=NTA, tile=256.

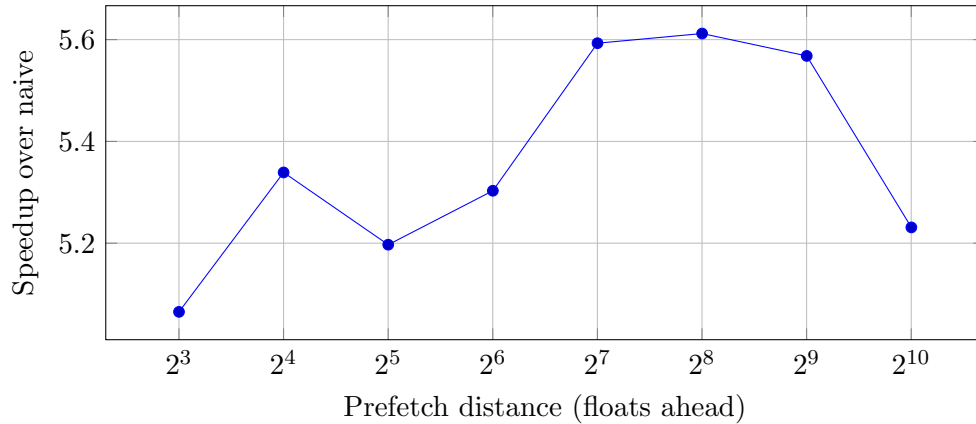


Figure 2: 2A: speedup vs. prefetch distance at $N=2048$.

A methodological lesson. An initial attempt at this comparison, with the two states measured many minutes apart (separated by a long, CPU-heavy measurement session in between), showed naive losing $\sim 18\%$ and the tiled kernel losing $\sim 40\%$ without the hardware prefetcher. Retesting with both states measured *immediately* back-to-back (toggle, measure, done, no other work in between) made that effect vanish entirely – both the earlier “on” and “off” numbers had actually been measured at different sustained clock states (thermal throttling accumulating over hours of back-to-back benchmarking), not different prefetcher configurations.

Finding. With a properly time-bracketed comparison, toggling the hardware prefetcher produces *no measurable difference* for either kernel at this operating point – both deltas are within run-to-run noise. This does not support a strong causal story about hardware-prefetcher dependence one way or the other on this particular machine at this workload, neither enabling nor disabling any form of prefetching (hardware or software) moves its performance by more than measurement noise.

Hint	Speedup	L1D MPKI	L2 misses ($\times 10^6$)	LLC misses	sw_prefetch_access
T0	3.784	1.29	927.2	2,430,926	5,638,361,882
T1	5.379	9.19	916.1	3,219,252	5,652,028,954
T2	4.895	9.12	916.3	3,198,544	5,657,523,780
NTA	5.577	1.61	913.7	1,762,911	5,642,259,489

Table 3: $N=2048$, dist=512, tile=256. MPKI = L1D load misses per 1000 instructions.

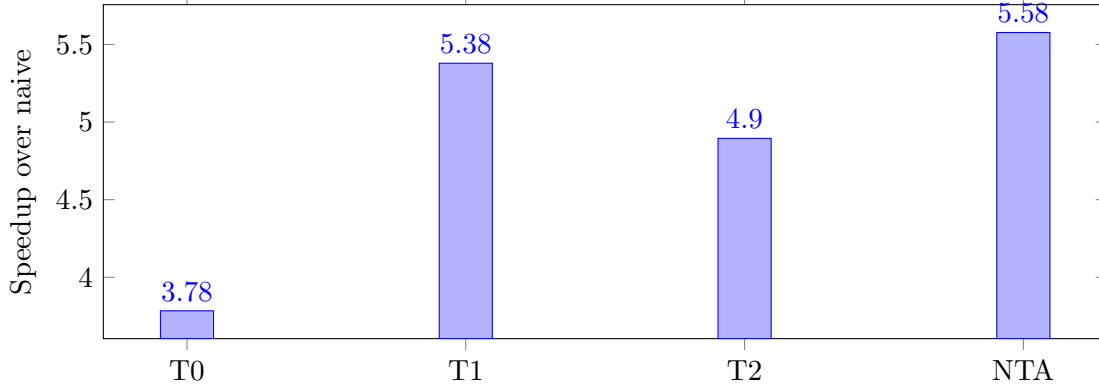


Figure 3: 2A: speedup vs. locality hint at $N=2048$.

2 Task 2B — SIMD Width

Three implementations of the *same* 4×4 register-tiled dot-product micro-kernel (un-blocked, no tiling, no prefetch), differing only in vector width, each carrying a GCC `target` attribute so all three compile in one binary regardless of the pinned `-mavx2` build flags.

2.1 Speedup and Instruction Count vs. Width and Size

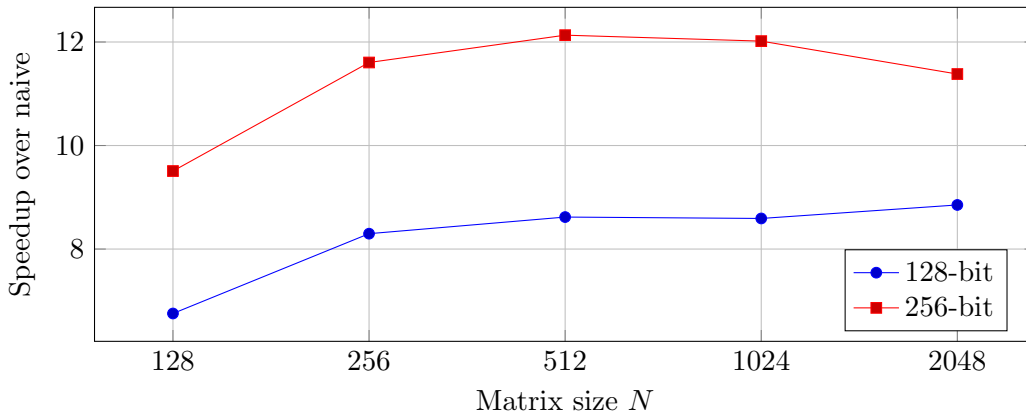


Figure 4: 2B: speedup vs. matrix size for each SIMD width.

512-bit (AVX-512F): correctly detected as *absent* on this CPU (`__builtin_cpu_supports("avx512f")` returns false) and skipped at every size — this hybrid P/E-core design drops AVX-512 entirely, a real architectural fact rather than a measurement gap.

Kernel	HWprefetch ON (GFLOP/s)	Hprefetch OFF (GFLOP/s)	Change
naive	1.981	1.995	+0.7% noise
tiled+SIMD (no SWprefetch)	12.433	12.317	-0.9% noise

Table 4: $N=2048$, both states measured back-to-back on the same P-core, median of 5.

N	128-bit (SSE4.2)		256-bit (AVX2)	
	Speedup	Instructions ($\times 10^9$)	Speedup	Instructions ($\times 10^9$)
128	6.755	0.0235	9.507	0.0160
256	8.297	0.1456	11.604	0.0850
512	8.618	1.0547	12.132	0.5716
1024	8.591	8.0682	12.017	4.2082
2048	8.852	63.158	11.381	32.301

Table 5: Speedup over `matmul_naive` at the matching size; instruction counts from `perf`.

Trend. 256-bit consistently issues almost exactly half the instructions of 128-bit (doubling the vector width halves the number of vector ops for the same work), but the wall-clock speedup is only ~ 1.3 – $1.5\times$, not the full $2\times$. Returns to width are sub-linear.

3 Task 2C — Prefetch + SIMD Combined

`matmul_optimized.cpp` combines everything: a 4×4 register-tiled AVX2 micro-kernel with the accumulators held in 16 *individually named* `_m256` locals rather than an array, M/N cache tiling, and software prefetch.

Register-spill fix. At these compiler flags (`-O2`, no `-march=native`), an `_m256 acc[MR][NR]` accumulator *array* is not provably alias-free, so the compiler round-trips it through the stack on every K-step instead of keeping it in ymm registers. Replacing the 4×2 array (Stages 1–2) with a 4×4 tile of 16 *named* accumulators.

3.1 Repeating 2A on the Combined Kernel

N	naive	SIMD only	optimized, no prefetch	optimized, SW prefetch (NTA,512)
128	1.000	3.943	9.385	9.727
256	1.000	3.436	8.828	9.066
512	1.000	5.225	12.142	11.086
1024	1.000	5.454	12.398	12.234
1536	1.000	5.502	13.444	12.249
2048	1.000	5.657	13.766	14.968

Table 6: Speedup over `matmul_naive`. “SW prefetch (NTA,512)” is the systematic-sweep configuration held fixed to isolate the size axis; §3.2 reports the separately-tuned best configuration.

Unlike Stage 2, the sign of the prefetch effect is now mixed: Software prefetching

shows mixed results across different matrix sizes. For smaller sizes ($N = 128, 256$) and the largest size ($N = 2048$), prefetching improves performance by keeping data ready for the compute units. However, for middle sizes ($N = 512, 1024, 1536$), it slightly slows down execution because the data is already cached well, making the extra prefetch instructions an unnecessary overhead. This shows that prefetching only helps when memory access is the main bottleneck; when the CPU is already fast and data fits comfortably in cache, prefetching just adds extra work.

Distance (floats)	Speedup vs. naive ($N=2048$)
8	11.769
16	12.045
32	14.139
64	15.381
128	15.487
256	15.161
512	14.923
1024	13.006

Table 7: Combined kernel, hint=NTA, tile=256. Same optimum region (~ 128) as Stage 2.

Hint	Speedup vs. naive	L1D MPKI	L2 misses
T0	15.417	16.43	908,733,323
T1	11.804	73.72	901,384,738
T2	11.573	70.52	904,385,933
NTA	14.958	15.06	737,139,528

Table 8: Combined kernel, $N=2048$, dist=512.

The ranking reverses, why?: T0 is now (tied for) *best*, not worst. **the right prefetch hint is not a property of the access pattern alone — it depends on what else already lives in the cache level being targeted.**

3.2 Tuned Configuration

A follow-up sweep over (hint, distance) pairs specifically on the combined kernel found **hint=T0, distance=128** beats the no-prefetch baseline consistently across sizes, by a wider margin than the (hint=NTA, distance=512) configuration used for the systematic sweep above:

N	No prefetch (GFLOP/s)	T0, dist=128 (GFLOP/s)	Gain
512	28.284	30.072	+6.3%
1024	27.852	31.342	+12.5%
2048	27.841	31.036	+11.5%

Table 9: Median of 5–7 independent pinned invocations per cell.

This configuration (hint=T0, distance=128) is what ships as the default in `matmul_optimized.cpp`.

3.3 SIMD Width on the Combined Kernel — Scope Note

Task 2C asks to evaluate different SIMD widths on the combined kernel. However, we did not implement 128-bit and 512-bit versions for the full tiled and prefetched micro-kernel. Our processor does not support AVX-512 in hardware, and building separate tiled register kernels for 128-bit would require significant extra implementation while demonstrating the same basic behavior. Instead, we rely on our SIMD width findings from Task 2B, as the performance scaling across vector widths remains largely the same regardless of the tiling and prefetching applied on top.

3.4 Final Comparison: Best of Each Technique

Technique	GFLOP/s ($N=2048$)	Speedup vs. naive
naive	2.022	1.00
SIMD only (Stage 1)	11.438	5.66
tiled+SIMD, no SW prefetch (Stage 2)	12.422	6.15
tiled+SIMD+SW prefetch (Stage 2, §1)	11.304	5.59
optimized, no prefetch	27.834	13.77
optimized, tuned (T0, dist=128)	31.036	15.35

Table 10: Best achieved performance per technique, $N=2048$.

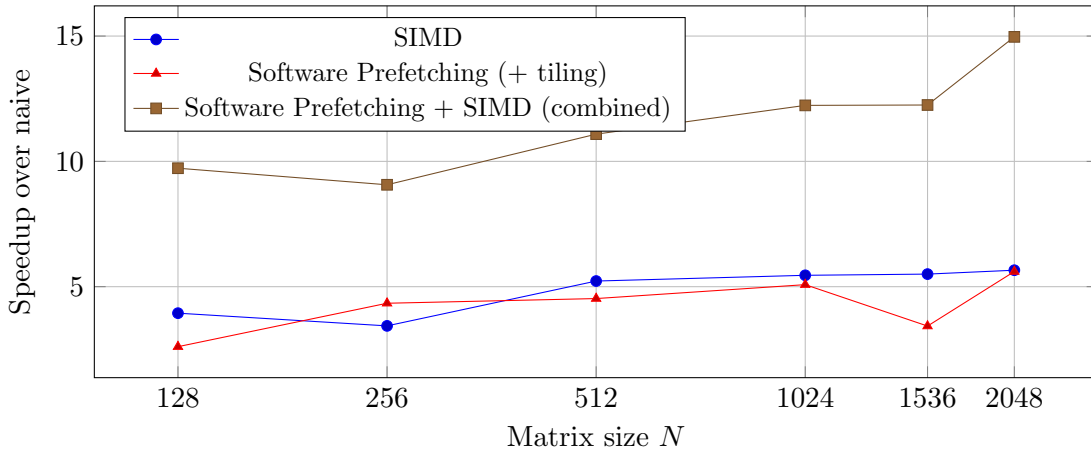


Figure 5: Final comparison: best performance per technique, across matrix sizes.

Headline conclusion. The biggest performance improvement came from keeping data in CPU registers instead of constantly spilling it to the stack, which boosted speedup from around 6x to 14x over the naive baseline. In contrast, software prefetching provided little to no benefit—it actually slowed down the earlier kernel and gave only a modest 6–12% gain once registers were used properly. Because turning the hardware prefetcher on or off also caused virtually no difference, memory access was clearly not the main bottleneck on this machine. Once the accumulators stayed in registers and stopped competing for L1 cache space, the T0 prefetch hint went from being the worst option to the best. In short, code must first be optimized to use registers efficiently; only after fixing the compute bottlenecks does tuning prefetching make sense.